

# 1 Analysis Basics

April 21, 2026

## 1 A First Program Analysis: Analysis at Character Level

We will start by treating source code as plain text files, without any further structural information. This obviously limits what we can do, but it provides a nice starting point for two different types of analyses.

### 1.1 Lines of Code

Measuring the complexity of software is important for many reasons. There are countless different complexity metrics, and while they all have weaknesses in how well they can actually capture “complexity”, they do have their use. The most common metric among all is to count the lines of code: The more lines a program has, the more complex it likely is. Let’s consider a simple Java example program:

```
[1]: code = """
package example;

public class Example {
    private int foo;

    public Example(int foo) {
        this.foo = foo;
    }

    public int getFoo() {
        return foo;
    }

    public int doBar(int bar) {
        if (bar > 0) {
            // This is the case where bar > 0
            return bar + foo;
        } else {
            return bar - foo;
        }
    }
}
"""
```

We've defined our program as a string so that we can process it using Python code.

```
[2]: for i, line in enumerate(code.split("\n")):
      print(str(i + 1).rjust(3, ' '), ': ', line)
```

```
1 :
2 : package example;
3 :
4 : public class Example {
5 :     private int foo;
6 :
7 :     public Example(int foo) {
8 :         this.foo = foo;
9 :     }
10 :
11 :     public int getFoo() {
12 :         return foo;
13 :     }
14 :
15 :     public int doBar(int bar) {
16 :         if (bar > 0) {
17 :             // This is the case where bar > 0
18 :             return bar + foo;
19 :         } else {
20 :             return bar - foo;
21 :         }
22 :     }
23 : }
24 :
```

To count the number of lines in the file, we just need to split the source code at newline characters:

```
[3]: len(code.split("\n"))
```

```
[3]: 24
```

Voila! Let's put this in a function.

```
[4]: def count_loc(code):
      lines = code.split("\n")
      return len(lines)
```

```
[5]: count_loc(code)
```

```
[5]: 24
```

Our LOC metric includes lines without actual code, such as empty lines, lines with comments, or lines with only braces. We can, however, easily filter these.

```
[6]: def count_loc(code):
    lines = code.split("\n")

    # Remove closing braces
    lines = [l.replace("}", "") for l in lines]

    # Remove trailing whitespaces
    lines = [l.strip() for l in lines]

    # Remove comment lines
    lines = [l for l in lines if not l.startswith("//")]

    # Remove empty lines
    lines = [l for l in lines if l]

    return len(lines)
```

```
[7]: count_loc(code)
```

```
[7]: 12
```

Let's have a look at the actual lines to see if this makes sense.

```
[8]: for i, line in enumerate(code.split("\n")):
    l = line.replace("}", "").strip()
    if l and not l.startswith("//"):
        print(str(i + 1).rjust(3, ' '), ':', l)
```

```
2 : package example;
4 : public class Example {
5 : private int foo;
7 : public Example(int foo) {
8 : this.foo = foo;
11 : public int getFoo() {
12 : return foo;
15 : public int doBar(int bar) {
16 : if (bar > 0) {
18 : return bar + foo;
19 : else {
20 : return bar - foo;
```

Of course, if there are multi-line comments, things become more tricky. However, we'll skip this for now and move on to the next analysis.

## 1.2 Code Clone Detection

Code clones are fragments of source code that occur repeatedly in identical or similar way. Code clones are considered a quality issue in programs, and Fowler's "Stink Parade of Bad Smells".

Code clones may exist for legitimate reasons: They are commonly used during development when

reusing existing code using copy&paste, which can speed up development. However, if there is a bug in the cloned code, then this creates a maintenance issue: When fixing a bug in one clone, one would need to fix the same bug in all clones.

Code clone detection is a program analysis that aims to identify locations of similar/identical code, such that the code quality can be improved by refactoring these code locations. Notably, similar techniques can also be used to identify plagiarism in programming assignments.

We distinguish four different types of code clones.

### 1.2.1 Type 1 Clones

Type 1 clones are identical code fragments, which may have some variations in whitespace, layout, and comments.

```
[9]: type_1_1 = """
public class Foo {
    public void foo(int x) {
        System.out.println("Hello Clone!");
        int j = 10;
        for(int i = 0; i < x; i++) {
            System.out.println("Another iteration");
        }
    }
}
"""
```

```
[10]: type_1_2 = """
public class Bar {
    public void bar(int x) {
        System.out.println("Hello Clone!");
        int j = 10;
        for(int i = 0; i < x; i++) {
            System.out.println("Another iteration");
        }
    }
}
"""
```

The method bodies of methods `foo` and `bar` are clearly identical, so this constitutes a type 1 clone. However, also the following method `zoo` is a type 1 clones since it only differs in whitespaces and indentation:

```
[11]: type_1_3 = """
public class Zoo
{
    public void zoo(int x)
    {
        System.out.println("Hello Clone!");
    }
}
```

```

        int j = 10;
        for(int i = 0; i < x; i++)
        {
            System.out.println("Another iteration");
        }
    }
}
"""

```

In our initial analysis we will only consider type 1 clones.

### 1.2.2 Type 2 Clones

Type 2 clones are syntactically equivalent fragments with some variations in identifiers, literals, types, whitespace, layout and comments. The method `bar` in the following snippet is a type 2 clone of the methods shown in our type 1 examples above: It uses different literals, strings, and variable names.

```

public class Bar {
    public void bar(int y) {
        System.out.println("Some other text");
        int z = 90;
        for(int j = 0; j < y; j++) {
            System.out.println("Yet some more different text");
        }
    }
}

```

### 1.2.3 Type 3 Clones

Type 3 clones are syntactically similar code fragments with inserted, deleted, or updated statements.

```

public class Bar {
    public void bar(int z) {
        System.out.println("Some completely different text");
        int x = 90;
        String s = "Another assignment";
        for(int j = 0; j < z; j++) {
            System.out.println("Yet some more different text");
        }
    }
}

```

This version of `bar` not only uses different literals and variable names, but it also includes an additional statement creating a variable `s`.

### 1.2.4 Type 4 Clones

Finally, type 4 clones are semantically equivalent, but syntactically different. This is difficult with the non-sensical code we've used above, so here's another example:

```
public class Bar {
    public String concatenate(String a, String b) {
        String result = "";
        result += a;
        result += b;
        return result
    }
}

public class Bar {
    public String concatenate(String a, String b) {
        StringBuffer buffer = new StringBuffer();
        buffer.append(a);
        buffer.append(b);
        return buffer.toString();
    }
}
```

There is very little syntactic similarity between the two versions of method `concatenate`, however they both semantically do exactly the same.

Overall, between 7%-23% of the code in a typical software system is cloned

### 1.2.5 Simple Type 1 Clone Detection Analysis

Code clone analysis has been a topic of research for decades, and there are numerous different techniques. We will use a variant of a classical approach defined in the following paper:

Ducasse, S., Rieger, M., & Demeyer, S. (1999, August). A language independent approach for detecting duplicated code. In Proceedings IEEE International Conference on Software Maintenance-1999 (ICSM'99). (pp. 109-118). IEEE.

We will focus only on type 1 clones today, but will tackle type 2 clones in the next lecture. We will also make another simplification: We will compare code line by line, so it will be possible to fool our analysis by wrapping lines differently. Again, this is something we will tackle next time.

At the core of the analysis lies a cross comparison of all the lines.

```
[12]: lines1 = type_1_1.split("\n")
      lines1
```

```
[12]: [' ',
      'public class Foo {',
      '    public void foo(int x) {',
      '        System.out.println("Hello Clone!");',
      '        int j = 10;',
      '        for(int i = 0; i < x; i++) {',
```

```
'        System.out.println("Another iteration");',
'    }',
' }',
'}',
'']
```

```
[13]: lines2 = type_1_2.split("\n")
lines2
```

```
[13]: ['',
'public class Bar {',
'    public void bar(int x) {',
'        System.out.println("Hello Clone!");',
'        int j = 10;',
'        for(int i = 0; i < x; i++) {',
'            System.out.println("Another iteration");',
'        }',
'    }',
'}',
'']
```

Since we compare every line in `lines1` with every line in `lines2`, we create a matrix that summarises these comparisons.

```
[14]: def compare_lines(lines1, lines2):
        matrix = []

        for line1 in lines1:
            row = []
            for line2 in lines2:
                row.append(1 if line1 == line2 else 0)

            matrix.append(row)

        return matrix
```

```
[15]: compare_lines(lines1, lines2)
```

```
[15]: [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
[0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
```

```
[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1]]
```

In this matrix, we can find clones if there are diagonals of 1s. We can spot one along the middle of the matrix, with 8 identical lines.

We thus need to look for these in our matrix. For each position in the matrix we check for the longest “block” of matches starting at the positions in the two code fragments starting at this position.

```
[16]: def get_block_at(matrix, x, y):
        block = []

        while (x < len(matrix) and y < len(matrix[x]) and matrix[x][y]):
            block.append((x, y))
            x += 1
            y += 1

        return block
```

In our example, we know the clone starts in line 3 of each of the fragments, so we can check if this is found correctly.

```
[17]: matrix = compare_lines(lines1, lines2)
        get_block_at(matrix, 3, 3)
```

```
[17]: [(3, 3), (4, 4), (5, 5), (6, 6), (7, 7), (8, 8), (9, 9), (10, 10)]
```

Indeed the two snippets are identical starting from line 3.

To find code clones, we extract all blocks from the matrix. We might want to restrict ourselves to blocks of a minimum size (e.g. at least 3 matching statements). Furthermore, we need to avoid collecting subsumed blocks: If we have identified the block starting at (3,3), then there is also a block starting at (4,4) in our example. Since we want to find the largest possible overlap, we keep track of what we have already identified and skip redundant checks.

```
[18]: def get_blocks(matrix, min_size = 3):
        blocks = []
        covered = set()

        width = len(matrix)
        height = len(matrix[0])

        for x in range(width):
            for y in range(height):
                if (x, y) in covered:
                    continue

                block = get_block_at(matrix, x, y)
                if len(block) >= min_size:
                    blocks.append(block)
```



```

        for (bx, by) in block:
            covered.add((bx, by))

    return blocks

```

Running this function on our example matrix should return a single block from line 3 to the end.

```
[19]: get_blocks(matrix)
```

```
[19]: [(3, 3), (4, 4), (5, 5), (6, 6), (7, 7), (8, 8), (9, 9), (10, 10)]
```

Let's pretty-print the relevant code.

```
[20]: clones = get_blocks(matrix)
for clone in clones:
    print("Code in snippet 1:")
    for i, j in clone:
        print(str(i + 1).rjust(3, ' '), ':', lines1[i])

    print("\nCode in snippet 2:")
    for i, j in clone:
        print(str(j + 1).rjust(3, ' '), ':', lines1[j])

```

Code in snippet 1:

```

4 :      System.out.println("Hello Clone!");
5 :      int j = 10;
6 :      for(int i = 0; i < x; i++) {
7 :          System.out.println("Another iteration");
8 :      }
9 :  }
10 : }
11 :

```

Code in snippet 2:

```

4 :      System.out.println("Hello Clone!");
5 :      int j = 10;
6 :      for(int i = 0; i < x; i++) {
7 :          System.out.println("Another iteration");
8 :      }
9 :  }
10 : }
11 :

```

The match includes the closing braces. Arguably, just like when counting the lines of code, these lines are not relevant. Let's also consider what happens when comparing against the second type 1 clone we defined earlier.

```
[21]: lines3 = type_1_3.split("\n")
lines3

```

```
[21]: [' ',
      'public class Zoo ',
      '{',
      '    public void zoo(int x) ',
      '    {',
      '        System.out.println("Hello Clone!");',
      '    ',
      '        int j = 10;',
      '        for(int i = 0; i < x; i++) ',
      '        {',
      '            System.out.println("Another iteration");',
      '        }',
      '    }',
      '}',
      '']
```

```
[22]: matrix2 = compare_lines(lines1, lines3)
      matrix2
```

```
[22]: [[1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0],
      [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0],
      [1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1]]
```

There is only one clone block in this matrix:

```
[23]: get_blocks(matrix2)
```

```
[23]: [[(8, 12), (9, 13), (10, 14)]]
```

Incidentally, that block only covers the last two closing braces and empty line:

```
[24]: clones2 = get_blocks(matrix2)
      for clone in clones2:
          print("Code in snippet 1:")
          for i, j in clone:
              print(str(i + 1).rjust(3, ' '), ':', lines1[i])

          print("\nCode in snippet 2:")
          for i, j in clone:
              print(str(j + 1).rjust(3, ' '), ':', lines3[j])
```

Code in snippet 1:

```
9 :   }
10 : }
11 :
```

Code in snippet 2:

```
13 :   }
14 : }
15 :
```

In order to make a cleaner comparison, we will need to apply the same filtering of lines as we did when counting lines of code.

```
[25]: def get_lines(code):
        lines = [l.replace("}", "").replace("{", "").strip() for l in code.
        ↪split("\n")]
        code_lines = [l for l in lines if l and not l.startswith("//")]

        return code_lines
```

```
[26]: get_lines(type_1_1)
```

```
[26]: ['public class Foo',
        'public void foo(int x)',
        'System.out.println("Hello Clone!");',
        'int j = 10;',
        'for(int i = 0; i < x; i++)',
        'System.out.println("Another iteration");']
```

```
[27]: def print_clones(code1, code2):
        lines1 = get_lines(code1)
        lines2 = get_lines(code2)

        matrix = compare_lines(lines1, lines2)
        clones = get_blocks(matrix)

        for clone in clones:
            print("Code in snippet 1:")
            for i, j in clone:
                print(str(i + 1).rjust(3, ' '), ':', lines1[i])

            print("Code in snippet 2:")
            for i, j in clone:
                print(str(j + 1).rjust(3, ' '), ':', lines2[j])
            print("\n")
```

With our refined clone detection function we can now also find the code clone between the first and third snippet.

```
[28]: print_clones(type_1_1, type_1_3)
```

Code in snippet 1:

```
3 : System.out.println("Hello Clone!");
4 : int j = 10;
5 : for(int i = 0; i < x; i++)
6 : System.out.println("Another iteration");
```

Code in snippet 2:

```
3 : System.out.println("Hello Clone!");
4 : int j = 10;
5 : for(int i = 0; i < x; i++)
6 : System.out.println("Another iteration");
```

We can also spot multiple clones.

```
[29]: type_1_4 = """
public class Bar {
    public void foo(int x) {
        System.out.println("Hello Clone!");
        int j = 10;
        for(int i = 0; i < x; i++) {
            System.out.println("Another iteration");
        }
    }

    public void bar(int x) {
        System.out.println("Hello Clone!");
        int j = 10;
        for(int i = 0; i < x; i++) {
            System.out.println("Another iteration");
        }
    }
}
"""
```

```
[30]: print_clones(type_1_1, type_1_4)
```

Code in snippet 1:

```
2 : public void foo(int x)
3 : System.out.println("Hello Clone!");
4 : int j = 10;
5 : for(int i = 0; i < x; i++)
6 : System.out.println("Another iteration");
```

Code in snippet 2:

```
2 : public void foo(int x)
3 : System.out.println("Hello Clone!");
4 : int j = 10;
```

```

5 : for(int i = 0; i < x; i++)
6 : System.out.println("Another iteration");

```

Code in snippet 1:

```

3 : System.out.println("Hello Clone!");
4 : int j = 10;
5 : for(int i = 0; i < x; i++)
6 : System.out.println("Another iteration");

```

Code in snippet 2:

```

8 : System.out.println("Hello Clone!");
9 : int j = 10;
10 : for(int i = 0; i < x; i++)
11 : System.out.println("Another iteration");

```

Our example `type_1_4` actually contains a clone between its two methods `foo` and `bar`. However, when looking for clones within a file we need to make sure we skip the trivial clone (the file is obviously identical to itself, so our matrix will contain a diagonal of 1s):

```
[31]: compare_lines(get_lines(type_1_4), get_lines(type_1_4))
```

```
[31]: [[1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0],
      [0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0],
      [0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0],
      [0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0],
      [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1],
      [0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0],
      [0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 0],
      [0, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0],
      [0, 0, 0, 0, 1, 0, 0, 0, 0, 1, 0],
      [0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 1]]
```

Besides the trivial diagonal, there's another noteworthy thing happening here: The matrix is of course symmetrical across the diagonal. When extracting clone blocks, we only need to consider half the matrix.

```
[32]: def get_blocks_within(matrix, min_size = 3):
      blocks = []
      covered = set()

      width = len(matrix)
      height = len(matrix[0])

      for x in range(width):
          for y in range(x, height):
              if (x, y) in covered:
```

```

        continue

    block = get_block_at(matrix, x, y)
    if len(block) >= min_size:
        blocks.append(block)
        for (bx, by) in block:
            covered.add((bx, by))

return blocks

```

Let's put this all together.

```

[33]: def print_clones(code):
    lines = get_lines(code)

    matrix = compare_lines(lines, lines)
    clones = get_blocks_within(matrix)

    for clone in clones:
        # Skip the trivial clone in the self comparison
        if len(clone) == len(lines):
            continue
        print("Code in snippet 1:")
        for i, j in clone:
            print(str(i + 1).rjust(3, ' '), ':', lines[i])

        print("Code in snippet 2:")
        for i, j in clone:
            print(str(j + 1).rjust(3, ' '), ':', lines[j])
        print("\n")

```

Now let's check for clones within our snippet with two methods.

```

[34]: print_clones(type_1_4)

```

```

Code in snippet 1:
 3 : System.out.println("Hello Clone!");
 4 : int j = 10;
 5 : for(int i = 0; i < x; i++)
 6 : System.out.println("Another iteration");
Code in snippet 2:
 8 : System.out.println("Hello Clone!");
 9 : int j = 10;
10 : for(int i = 0; i < x; i++)
11 : System.out.println("Another iteration");

```

Our algorithm cannot detect type 2 clones yet – in order to do so it would need to ignore differences

in literals, strings, and variable names. We will have a look how to achieve this in the next lecture, when we move from considering source code at the character/line level to *tokens*.